

06

Uslovni testovi

Programi moraju da donose odluke: šta se dešava sa sekvencom zavisi od njenih osobina

Uslovi: True ili False

```
print(3 == 5)
print(3 > 5)
print(3 <= 5)
print(len("ATGC") > 5)
print("GAATTC".count("T") > 1)
print("ATGCTT".startswith("ATG"))
print("ATGCTT".endswith("TTT"))
print("ATGCTT".isupper())
print("ATGCTT".islower())
print("V" in ["V", "W", "L"])
```

```
False
False
True
False
True
True
False
True
False
True
```

Operator	Značenje
<code>==</code>	jednako (dva znaka =)
<code>!=</code>	različito
<code>> <</code>	veće, manje
<code>>= <=</code>	veće ili jednako, manje ili jednako
<code>in</code>	vrednost je u listi

`True` i `False` su posebne ugrađene vrednosti: velikim početnim slovom i bez navodnika. Mnogi metodi ih vraćaju, npr. `startswith`.

if: blok se izvršava samo ako je uslov tačan

```
expression_level = 125
if expression_level > 100:
    print("gene is highly expressed")
```

Reč **if**, uslov, dvotačka i uvučeni blok. Ovaj primer je beskoristan, jer se vrednost nikad ne menja: uslove pišemo za vrednosti koje ne znamo unapred.

```
accs = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72']
for accession in accs:
    if accession.startswith('a'):
        print(accession)
```

```
ab56
ay93
ap97
```

Linija u if-u unutar petlje uvučena je dvaput: za petlju i za if.

else: jedno ili drugo

Dve grane

```
expression_level = 125
if expression_level > 100:
    print("gene is highly expressed")
else:
    print("gene is lowly expressed")
```

Izvršava se tačno jedna od dve grane; ovde prva.

Podela u dva fajla

```
file1 = open("one.txt", "w")
file2 = open("two.txt", "w")
for accession in accs:
    if accession.startswith('a'):
        file1.write(accession + "\n")
    else:
        file2.write(accession + "\n")
```

one.txt: ab56, ay93, ap97 · two.txt: bh84, hv76, bd72

else nema svoj uslov: njegov blok se izvršava kad blok if nije. if i else stoje na istom nivou uvlačenja.

Tri grane: ugnežđeni if ili elif

if unutar else

```
for accession in accs:
    if accession.startswith('a'):
        file1.write(accession + "\n")
    else:
        if accession.startswith('b'):
            file2.write(accession + "\n")
        else:
            file3.write(accession + "\n")
```

Radi, ali je teško za čitanje: svaka nova grana dodaje još jedan nivo uvlačenja.

elif = else + if

```
for accession in accs:
    if accession.startswith('a'):
        file1.write(accession + "\n")
    elif accession.startswith('b'):
        file2.write(accession + "\n")
    else:
        file3.write(accession + "\n")
```

Koliko god grana, uvek samo jedan dodatni nivo uvlačenja.

if, elif i else

```
# file1, file2, file3: otvoreni za pisanje
accs = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72']
for accession in accs:
    if accession.startswith('a'):
        file1.write(accession + "\n")
    elif accession.startswith('b'):
        file2.write(accession + "\n")
    else:
        file3.write(accession + "\n")
```

if

blok se izvršava samo ako je uslov tačan

elif

nova grana bez novog nivoa uvlačenja

else

nema svoj uslov: izvršava se kad nijedna grana iznad nije

REZULTAT

```
one.txt:  ab56 ay93 ap97
two.txt:  bh84 bd72
three.txt: hv76
```

Složeni uslovi: and, or, not

Za svaki `acc` iz liste `['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72']` ispisujemo ga ako važi uslov:

```
acc.startswith('a') and acc.endswith('3')
```

ay93

```
acc.startswith('a') or acc.startswith('b')
```

ab56 bh84 ay93 ap97 bd72

```
(acc.startswith('a') or acc.startswith('b')) and acc.endswith('4')
```

bh84

```
acc.startswith('a') and not acc.endswith('6')
```

ay93 ap97

`while` ponavlja blok dok je uslov tačan. Pošto je `for` vrlo moćan, `while` se u Pythonu retko koristi.

while: ponavlja dok je uslov tačan

```
count = 0
while count < 10:
    print(count)
    count = count + 1
```

Isto, kraće, sa petljom for:

```
for count in range(10):
    print(count)
```

0
1
2
3
4
5
6
7
8
9

for ili while?

for se ponavlja jednom za svaki element. while se ponavlja dok uslov ne postane netačan, pa broj krugova ne mora biti poznat unapred.

Petlja for je u Pythonu toliko moćna da se while koristi retko.

Funkcije koje vraćaju True ili False

Da li je sekvenca bogata sa AT?

```
def is_at_rich(dna):  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    if at_content > 0.65:  
        return True  
    else:  
        return False
```

Kraće: vrati sam uslov

```
def is_at_rich(dna):  
    # ... računanje kao levo  
    return at_content > 0.65
```

```
print(is_at_rich("ATTATCTACTA"))  
print(is_at_rich("CGGCAGCGCT"))
```

True
False

Takva funkcija ide direktno u uslov: `if is_at_rich(my_dna):`

Korak 1: linija postaje kolone

Svaka linija: vrsta, sekvenca, ime gena i nivo ekspresije, razdvojeni zarezima.

```
data = open("data.csv")
for line in data:
    columns = line.rstrip("\n").split(",")
    species = columns[0]
    sequence = columns[1]
    name = columns[2]
    expression = columns[3]
    print(name)
```

pogl. 3 otvaranje fajla
pogl. 4 linija po linija; split u kolone
pogl. 6 uslov: ispisati ili ne

```
kdy647
jdg766
kdy533
hdt739
hdu045
teg436
```

IZLAZ

Vrsta i dužina sekvence

Geni iz *D. melanogaster* ili *D. simulans*

```
# ... čitanje kolona kao u koraku 1, pa:  
if species == "Drosophila melanogaster" or species == "Drosophila simulans":  
    print(name)
```

```
kdy647  
jdg766  
kdy533
```

Geni dužine između 90 i 110 baza

```
# ... čitanje kolona kao u koraku 1, pa:  
if len(sequence) > 90 and len(sequence) < 110:  
    print(name)
```

Dužinu računamo sa **len**; oba dela uslova moraju da važe, pa ih spajamo sa **and**.

AT sadržaj i ekspresija: pazite na tip

```
# get_at_content iz poglavlja 5, na početku programa  
# ... čitanje kolona kao u koraku 1, pa:  
    if get_at_content(sequence) < 0.5 and expression > 200:  
        print(name)
```

`TypeError: '>' not supported between instances of 'str' and 'int'`

Sve kolone pročitane iz fajla su stringovi, pa i `expression`. String ne može da se poredi sa brojem 200.

```
if get_at_content(sequence) < 0.5 and int(expression) > 200:  
    print(name)
```

Rešenje: `int(expression)`.

Složen uslov iz tri dela

```
(name.startswith('k') or name.startswith('h'))
```

ime počinje sa k ili sa h

and

```
species != "Drosophila melanogaster"
```

vrsta nije D. melanogaster

```
# ... čitanje kolona kao u koraku 1, pa:  
if (name.startswith('k') or name.startswith('h')) and species != "Drosophila melanogaster":  
    print(name)
```

Isti uslov piše se i sa **not**: `and not species == "Drosophila melanogaster"`. Zagrade oko dela sa **or** su obavezne.

Visok, nizak ili srednji AT sadržaj

```
data = open("data.csv")
for line in data:
    columns = line.rstrip("\n").split(",")
    species = columns[0]
    sequence = columns[1]
    name = columns[2]
    expression = columns[3]
    if get_at_content(sequence) > 0.65:
        print(name + " has high AT content")
    elif get_at_content(sequence) < 0.45:
        print(name + " has low AT content")
    else:
        print(name + " has medium AT content")
```

`data.csv`: jedan gen po liniji, a polja su razdvojena zarezima: vrsta, sekvenca, ime gena, nivo ekspresije.

IZLAZ

```
kdy647 has high AT content
jdg766 has medium AT content
kdy533 has medium AT content
hdt739 has low AT content
hdu045 has medium AT content
teg436 has medium AT content
```

`get_at_content` je funkcija iz poglavlja 5. Linija iz fajla postaje lista kolona: `rstrip` pa `split`.

Rezime i vežbe

Šta smo naučili

- Uslovi daju True ili False
- Operatori ==, !=, <, >, <=, >= i in
- if i else često, elif ponekad, while retko
- and, or, not i zagrade
- Funkcije koje vraćaju True ili False

Vežbe: imena gena iz data.csv

1. iz vrsta Drosophila melanogaster ili Drosophila simulans
2. dužine između 90 i 110 baza
3. sa AT sadržajem manjim od 0.5 i ekspresijom većom od 200
4. koja počinju sa k ili h, osim gena iz D. melanogaster
5. uz poruku da li je AT sadržaj visok (> 0.65), nizak (< 0.45) ili srednji